REVERSE ENGINEERING & ASSEMBLY LANGUAGE

A Complete Study Guide for Ethical Cybersecurity Students

CTF Challenges • Web & App Analysis • Auth Bypass • Binary Analysis • Tools & Workflows

Chapter 01

Introduction to Reverse Engineering

1.1 What is Reverse Engineering?

Reverse engineering (RE) is the process of analysing a compiled binary, web application, firmware, or protocol to understand its internal workings without access to the original source code. In cybersecurity, RE is used to discover vulnerabilities, analyse malware, recover lost source code, and solve CTF challenges.

Unlike forward engineering (design → code → product), RE starts from the end artefact and works backwards: binary → disassembly → pseudo-code → logic understanding.

1.2 Ethical & Legal Framework

- Only test systems you **own** or have **explicit written authorisation** to test.
- CTF platforms (PicoCTF, HackTheBox, TryHackMe, pwn.college) provide legal sandboxes.
- Bug-bounty programmes explicitly permit security research within defined scope.
- In many jurisdictions the Computer Misuse Act, CFAA (US), or similar laws make unauthorised access a criminal offence regardless of intent.
- Document everything: scope, methods, findings, and remediation recommendations.

1.3 Types of Reverse Engineering

- **Binary RE:** Analysing compiled executables (ELF, PE, Mach-O).
- **Web RE:** Examining JavaScript, APIs, cookies, and session tokens.
- **Mobile RE:** Decompiling APKs (Android) or IPAs (iOS).
- **Protocol RE:** Capturing and decoding network traffic.
- **Firmware RE:** Extracting and analysing embedded device firmware.
- **Malware Analysis:** Static and dynamic study of malicious software.

1.4 Core Skill Areas

Effective reverse engineering demands competency across several domains: low-level programming (C, ASM), operating-system internals, networking, and scripting (Python). Treat each CTF challenge as a micro-lab — each one sharpens a specific skill.

✦ Review Questions

1. Define reverse engineering and explain why it is valuable in cybersecurity.
2. What is the key ethical requirement before performing reverse engineering on any system?
3. List four types of reverse engineering and give a practical use case for each.
4. What legislation in your country governs unauthorised computer access?
5. Why are CTF platforms considered ideal learning environments for RE skills?

Chapter 02

The Reverse Engineering Workflow

2.1 The Five-Phase RE Workflow

A structured workflow prevents wasted time and missed artefacts. Every professional RE engagement follows roughly these five phases:

Phase	Name	Key Activities
1	Reconnaissance	Gather info: file type, size, hashes, strings, metadata, headers
2	Static Analysis	Disassemble / decompile without executing; examine imports, symbols
3	Dynamic Analysis	Run in sandbox; trace syscalls, memory, network; set breakpoints
4	Exploitation / Flag Extraction	Craft inputs, patch bytes, bypass checks, extract secrets
5	Documentation	Record methods, screenshots, PoC scripts, remediation advice

2.2 Reconnaissance — First Contact

- **file** command: identifies file type from magic bytes (not extension).
- **strings**: dumps printable ASCII/Unicode — often reveals flags, passwords, URLs.
- **xxd / hexdump**: raw hex view; identify file signatures (magic bytes).
- **sha256sum / md5sum**: hash for VirusTotal lookups or duplicate detection.
- **exiftool**: extracts embedded metadata from images, PDFs, Office files.
- **binwalk**: detects and extracts embedded files within firmware/binaries.

```
$ file challenge.bin
$ strings -n 8 challenge.bin | grep -i flag
$ xxd challenge.bin | head -20
$ exiftool image.png
```

2.3 Triage and Prioritisation

Before deep analysis, quickly triage: Does the binary call crypto functions? Does it open a network socket? Does it read environment variables? Each answer narrows your attack surface.

- Check imports/exports: what external libraries does it rely on?
- Look at the entry point and main function first.
- Search for interesting strings: 'password', 'token', 'flag', 'secret', 'key'.
- Identify anti-debugging techniques early (IsDebuggerPresent, ptrace checks).

✦ Review Questions

1. List the five phases of the RE workflow and describe the goal of each phase.
2. What does the 'file' command use to determine file type, and why is this more reliable than extensions?
3. Why should 'strings' be one of the first tools run on a binary?
4. What is triage in the context of reverse engineering, and why is it important?
5. What kind of information would you expect to find in binary imports/exports?

Chapter 03

Assembly Language Fundamentals

3.1 Why Assembly Matters in RE

Compilers translate high-level code into machine code. Disassemblers reverse this process, producing Assembly (ASM). Understanding ASM lets you read any program regardless of the original language. x86-64 (Intel/AMD) is the dominant architecture on modern desktops and servers.

3.2 x86-64 Registers

Registers are the CPU's fastest storage. The x86-64 architecture provides 16 general-purpose 64-bit registers:

Register	Size Variants	Common Purpose
RAX	RAX / EAX / AX / AL	Return value from functions, arithmetic accumulator
RBX	RBX / EBX / BX / BL	Base register; preserved across calls (callee-saved)
RCX	RCX / ECX / CX / CL	Counter (loops); 4th argument (Linux/Windows)
RDX	RDX / EDX / DX / DL	3rd argument; I/O operations; high part of mul
RSI	RSI / ESI	Source index; 2nd argument (Linux)
RDI	RDI / EDI	Destination index; 1st argument (Linux)
RSP	RSP / ESP	Stack pointer — top of current stack frame
RBP	RBP / EBP	Base pointer — bottom of current stack frame
RIP	RIP	Instruction pointer — address of next instruction
R8–R15	R8–R15 / R8D–R15D	Extended general-purpose; arguments 5–8 (Linux)
RFLAGS	RFLAGS / EFLAGS	Status flags: ZF, CF, SF, OF, PF

3.3 Key Instruction Groups

Data Movement

- **mov dst, src** — copy value from src to dst (most common instruction).
- **lea dst, [addr]** — load effective address (compute pointer arithmetic).
- **push reg** — decrement RSP by 8, write reg to [RSP].
- **pop reg** — read [RSP] into reg, increment RSP by 8.
- **xchg a, b** — swap two operands atomically.

Arithmetic & Logic

- **add/sub dst, src** — addition / subtraction, sets flags.
- **mul/imul** — unsigned / signed multiply.
- **div/idiv** — unsigned / signed divide (quotient in RAX, remainder in RDX).
- **and/or/xor/not** — bitwise operations. XOR reg,reg = zero a register fast.
- **shl/shr dst, n** — shift left/right by n bits (equivalent to mul/div by 2^n).

Control Flow

- **cmp a, b** — subtracts b from a, discards result, sets flags (ZF, SF, CF, OF).

- **test a, b** — bitwise AND, sets ZF if result is zero (often 'test eax,eax').
- **jmp addr** — unconditional jump to address.
- **jz / je addr** — jump if Zero Flag set (result was zero / operands equal).
- **jnz / jne addr** — jump if Zero Flag clear.
- **jg / jl / jge / jle** — signed comparisons.
- **call func** — push return address, jmp to func.
- **ret** — pop return address, jmp to it.

3.4 The Stack Frame

Every function call creates a stack frame. Understanding the prologue and epilogue is critical for following program flow and finding return addresses (buffer overflow pivot).

```
push rbp          ; save caller's base pointer
mov  rbp, rsp     ; set up new frame
sub  rsp, 0x30    ; allocate 48 bytes local storage
...               ; function body
mov  rsp, rbp     ; or 'leave'
pop  rbp
ret               ; jump back to caller
```

3.5 Calling Conventions

Linux x86-64 (System V ABI): Arguments passed in RDI, RSI, RDX, RCX, R8, R9; rest on stack. Return value in RAX. Windows x64: RCX, RDX, R8, R9; shadow space of 32 bytes reserved on stack. Knowing which register holds which argument lets you instantly understand what a function is doing.

✦ Review Questions

1. What register holds the return value of a function call on x86-64 Linux?
2. Explain the difference between 'cmp' and 'test' instructions.
3. What is the purpose of 'push rbp' / 'mov rbp, rsp' at the start of a function?
4. What does 'xor eax, eax' accomplish and why is it commonly used?
5. List the first four argument registers for Linux x86-64 System V ABI.

Chapter 04

Static Analysis

4.1 Definition and Goals

Static analysis examines a binary without executing it. This is safer (no risk of detonating malware) and allows thorough inspection of all code paths, including those rarely triggered at runtime.

4.2 Disassembly vs Decompilation

- **Disassembly:** Translates machine bytes back into Assembly mnemonics. Output is 1:1 with machine code. Tools: objdump, ndisasm, Ghidra, IDA Pro.
- **Decompilation:** Attempts to reconstruct high-level C-like pseudo-code from ASM. Output is an approximation — variable names are invented. Tools: Ghidra, IDA Hex-Rays, Binary Ninja, RetDec, Cutter.

Modern CTF solvers use Ghidra's decompiler as a first pass, then drop into ASM view when the decompiler is ambiguous.

4.3 Analysing Imports and Exports

Imports reveal what OS/library functions the binary calls — the best single clue to its behaviour before reading a single instruction:

- 'strcmp', 'strncmp', 'memcmp' → password or flag comparison.
- 'socket', 'connect', 'recv', 'send' → network communication.
- 'CreateFile', 'ReadFile', 'WriteFile' → file I/O (Windows).
- 'dlopen', 'dlsym' → dynamic library loading (possible anti-analysis).
- 'ptrace' → self-debugging anti-analysis trick.
- 'IsDebuggerPresent', 'CheckRemoteDebuggerPresent' → Windows anti-debug.

4.4 Control Flow Graphs (CFG)

A CFG visually represents all possible execution paths through a function. Each node is a basic block (sequential instructions with no branches). Edges represent jumps. In Ghidra and IDA, CFGs are automatically rendered per function. A heavily tangled CFG often indicates obfuscation.

4.5 Pattern Recognition in Disassembly

Common high-level constructs map to predictable ASM patterns:

if / else

```
cmp eax, 0x1337     ; compare to expected value
jne .fail           ; jump to fail if not equal
; success path here
```

for loop

```
mov ecx, 0          ; i = 0
.loop: cmp ecx, 10  ; while i < 10
jge .end
...                 ; body
```

```
inc ecx             ; i++
jmp .loop
```

strcmp equivalent

```
lea rdi, [rbp-0x20]  ; user input
lea rsi, [rip+0x200] ; hardcoded string
call strcmp
test eax, eax        ; if eax == 0 → strings equal
jnz .fail
```

4.6 Finding Hardcoded Secrets

Many beginner CTF binaries store the flag or a password comparison value in the .rodata section. Use Ghidra's 'Search → Program Text' or 'Defined Strings' to list all embedded strings instantly. Look for base64-encoded data, hex strings, or strings referencing 'flag', 'key', 'pass', 'ctf'.

✦ Review Questions

1. What is the difference between disassembly and decompilation?
2. Why is import analysis useful before reading assembly code?
3. What is a Control Flow Graph and how does it aid reverse engineering?
4. Write the x86-64 ASM pattern you would expect for a simple 'if (input == 0x1337)' check.
5. Where in a binary's section layout are hardcoded strings typically stored?

Chapter 05

Dynamic Analysis & Debugging

5.1 Why Dynamic Analysis?

Static analysis has limits: packed/encrypted code is unreadable until unpacked at runtime. Dynamic analysis executes the target in a controlled environment, letting you observe actual register values, memory contents, and system calls at each step.

5.2 GDB — The GNU Debugger

GDB is the standard Linux debugger. With the PEDA, pwndbg, or GEF plugins it becomes a powerful RE platform:

- **break *0xADDR** — set breakpoint at address.
- **break function_name** — break at function entry.
- **run [args]** — start execution.
- **continue (c)** — resume until next breakpoint.
- **next (n)** — step over (don't enter function calls).
- **step (s)** — step into function calls.
- **nexti / stepi** — step one machine instruction.
- **info registers** — display all register values.
- **x/20xg $rsp** — examine 20 64-bit hex values at RSP (stack view).
- **x/s 0xADDR** — examine memory as string.
- **set {int}0xADDR = 0x1** — patch memory at runtime.
- **disas function** — disassemble a function.
- **finish** — run until current function returns, show return value.

```
$ gdb ./challenge
(gdb) break main
(gdb) run AAAA
(gdb) info registers
(gdb) x/s $rdi
(gdb) nexti
```

5.3 Tracing System Calls (strace)

strace intercepts every Linux system call made by a process — file opens, network connections, memory mappings. Invaluable for understanding what a binary does at OS level without reading a line of ASM:

```
$ strace ./challenge 2>&1 | grep -E 'open|read|write|connect'
```

5.4 Library Call Tracing (ltrace)

ltrace intercepts calls to shared library functions (e.g., libc). It shows arguments and return values, making strcmp bypass trivial:

```
$ ltrace ./challenge myguess
strcmp("myguess", "supersecret") = -1   ; reveals target string!
```

5.5 Sandbox Environments

- Run unknown binaries in isolated VMs or Docker containers.
- Any.run, Cuckoo Sandbox, and CAPE provide automated dynamic analysis reports.
- Use snapshots — restore to a clean state after each run.
- Monitor file system changes: inotifywait, Procmon (Windows).

5.6 Patching Binaries

Sometimes the quickest solution is to patch the binary — flip a conditional jump to force the success path. In Ghidra: right-click instruction → Patch Instruction. In raw hex: locate the jump opcode and replace 'jnz' (0x75) with 'jmp' (0xEB) or NOP (0x90).

```
Original : 75 0A    ; jnz +10  (fail path)
Patched  : 90 90    ; NOP NOP  (fall through to success)
```

✦ Review Questions

1. What is the GDB command to display all current register values?
2. How does ltrace reveal a hardcoded password without disassembly?
3. Explain the difference between 'step' and 'next' in GDB.
4. What opcode would you use to NOP out a conditional jump in x86?
5. Why should unknown binaries always be run in a sandbox environment?

Chapter 06

Metadata Analysis

6.1 What is Metadata?

Metadata is data about data — information embedded in files that describes their origin, structure, or content. In CTF challenges, flags are frequently hidden in metadata. In real-world investigations, metadata reveals author identity, timestamps, GPS coordinates, and software versions.

6.2 File Metadata with ExifTool

ExifTool reads metadata from 150+ file formats: images (JPEG, PNG, TIFF, HEIC), documents (PDF, DOCX, XLSX), audio, video, and executables.

```
$ exiftool image.jpg
$ exiftool -GPS* photo.jpg      ; GPS coordinates only
$ exiftool -Comment *.png       ; Comment field all PNGs
$ exiftool -all= clean.jpg      ; Strip all metadata
```

- **GPS coordinates** can reveal a photographer's location.
- **Author/Creator** fields reveal user account names.
- **Software** field reveals editing tools, potential version exploits.
- **Timestamps** can prove (or disprove) file creation time claims.
- **Comment / Description** fields are common CTF flag hiding spots.

6.3 Binary File Headers and Magic Bytes

Every file format starts with a magic byte sequence. Knowing these lets you identify files even when renamed or extension-stripped:

Magic Bytes (Hex)	File Type	ASCII Signature
FF D8 FF	JPEG Image	ÿØÿ
89 50 4E 47 0D 0A 1A 0A	PNG Image	.PNG....
25 50 44 46	PDF Document	%PDF
50 4B 03 04	ZIP / DOCX / APK	PK..
7F 45 4C 46	ELF Linux Binary	.ELF
4D 5A	PE Windows Binary	MZ
47 49 46 38	GIF Image	GIF8
52 49 46 46	WAV/AVI	RIFF
1F 8B 08	GZIP Archive	...
4F 67 67 53	OGG Audio/Video	OggS

6.4 Steganography and Hidden Data

Steganography hides data inside other data. Images are the most common carrier in CTFs:

- **LSB Steganography:** Flag bits hidden in the least-significant bits of pixel RGB values. Tool: zsteg, stegsolve.

- **EOF Data:** Data appended after the file's logical end — binwalk detects this.
- **Alternate Data Streams (Windows NTFS):** Hidden data streams attached to files.
- **Audio steganography:** Data in spectrogram — open in Audacity, view as spectrogram.
- **PDF hidden layers:** Text hidden under images or set to white colour.

```
$ zsteg -a image.png
$ stegsolve image.png          ; GUI analysis tool
$ binwalk -e suspicious.jpg    ; extract appended data
$ foremost -i image.jpg        ; carve embedded files
```

✦ Review Questions

1. What ExifTool command would extract only GPS metadata from a JPEG?
2. What are magic bytes and why are they more reliable than file extensions?
3. Name three common metadata fields that could reveal a file author's identity.
4. What is LSB steganography and which tool is commonly used to detect it?
5. How would you detect data hidden after the EOF of a JPEG image?

Chapter 07

Web Application Reverse Engineering

7.1 The Web RE Mindset

Web applications expose far more surface area than binaries because the client-side code (HTML, CSS, JavaScript) is downloaded by the browser and therefore readable. The challenge is that production code is minified, obfuscated, and split across dozens of files. A methodical reconnaissance approach cuts through the noise.

7.2 Recon: What to Collect First

- **View Source (Ctrl+U):** Reveals HTML comments, hardcoded credentials, API endpoints.
- **DevTools → Sources:** All loaded JS/CSS files; set breakpoints, inspect call stack.
- **DevTools → Network:** Every HTTP request/response — headers, bodies, tokens, cookies.
- **DevTools → Application:** localStorage, sessionStorage, cookies, IndexedDB.
- **robots.txt:** Often reveals hidden paths the admin did not want indexed.
- **sitemap.xml:** Full site structure.
- **/.git/ directory:** Exposed git repos leak full source history.
- **/.env files:** Environment files may contain API keys, DB passwords.

7.3 JavaScript Deobfuscation

Modern JS obfuscators (javascript-obfuscator, UglifyJS, Obfuscator.io) use techniques like variable renaming, string splitting, eval() wrappers, and control flow flattening. Deobfuscation workflow:

- Step 1 — **Beautify:** Use js-beautify, DevTools 'Pretty Print' ({}), or https://prettier.io.
- Step 2 — **Rename variables:** Rename cryptic _0x1a2b to meaningful names as you analyse.
- Step 3 — **Decode strings:** Base64, hex, and Unicode escape sequences are common. Run encoded strings through console.log() or CyberChef.
- Step 4 — **Evaluate eval():** Log the argument to eval() instead of executing it; this reveals the hidden inner code.
- Step 5 — **Remove dead code:** Obfuscators inject junk branches; identify and mentally skip them.

```
// Replace eval with console.log to reveal hidden code:
// Original:  eval(atob('ZnVuY3Rpb24gY2hlY2soKXs...'))
// Modified: console.log(atob('ZnVuY3Rpb24gY2hlY2soKXs...'))
```

7.4 API Analysis with Burp Suite

Burp Suite Community Edition is a proxy that intercepts all HTTP/HTTPS traffic between your browser and a web application. Essential RE workflow:

- Configure browser to proxy through 127.0.0.1:8080.
- Install Burp's CA certificate to decrypt HTTPS.
- Use Intercept to pause and modify requests before they reach the server.
- Use Repeater to manually replay and tweak individual requests.
- Use Decoder to convert between base64, URL encoding, hex.
- Use Scanner (Pro) or manually test for IDOR, SQLi, auth bypass parameters.

7.5 Inspecting JWTs (JSON Web Tokens)

JWTs are three base64-encoded sections: Header.Payload.Signature. They carry session state and are commonly attacked in CTFs:

- **Algorithm confusion (alg:none):** Server accepts unsigned token if it doesn't validate the algorithm field. Set alg to 'none', remove the signature.
- **Weak secret brute-force:** If HS256, the signature is HMAC-SHA256 of header+payload using a secret. Tools: hashcat, jwt_tool can brute-force weak secrets.
- **RS256 → HS256 confusion:** If server uses public key as HMAC secret.

```
# Decode JWT manually:
echo 'eyJhbGciOiJIUzI1NiJ9' | base64 -d
# Output: {"alg":"HS256"}
```

7.6 Cookie and Session Analysis

- Inspect Set-Cookie headers: Secure, HttpOnly, SameSite flags.
- Decode base64-encoded cookies — they may contain serialised objects.
- Check for predictable session IDs (sequential, time-based).
- Look for 'remember me' tokens stored in localStorage — often less protected.

✦ Review Questions

1. Name three browser DevTools panels useful for web RE and state what each reveals.
2. What is the four-step workflow for JavaScript deobfuscation?
3. How would you intercept and modify an HTTPS request using Burp Suite?
4. Explain the 'alg:none' JWT attack and what server misconfiguration enables it.
5. What sensitive information might be found in a /.git/ directory exposed on a web server?

Chapter 08

Authentication Bypass (Educational)

■ All techniques in this chapter are for CTF challenges, authorised penetration testing, and academic understanding only. Never apply to systems without explicit written permission.

8.1 SQL Injection Auth Bypass

Classic SQL injection exploits improper input sanitisation in login queries. The backend SQL looks like:

```
SELECT * FROM users WHERE username='$user' AND password='$pass'
```

By injecting SQL metacharacters, the logic can be subverted:

```
Username: admin'--
Query becomes: SELECT * FROM users WHERE username='admin'--' AND password='...'
The -- comments out the password check → login as admin with no password.
```

- Single-quote test: enter a single quote — if the app errors, injection may be possible.
- Classic bypass payloads: ' OR '1'='1 ; ' OR 1=1-- ; admin'--
- Use sqlmap for automated detection and exploitation on authorised targets.

8.2 Parameter Tampering

Web applications frequently pass user roles, IDs, or permissions as HTTP parameters, cookies, or hidden form fields — and trust them without server-side verification:

- **Horizontal privilege escalation:** Change user_id=42 to user_id=43 to access another user's data (IDOR).
- **Vertical privilege escalation:** Change role=user to role=admin in a cookie or request body.
- **Price manipulation:** Modify price=99.99 to price=0.01 in a checkout request.

8.3 Insecure Direct Object Reference (IDOR)

IDOR occurs when an object reference (ID, filename, UUID) is directly exposed and the server doesn't verify ownership. To find IDORs:

- Enumerate numeric IDs (/api/user/1, /api/user/2, ...).
- Swap your session token while requesting another user's resource.
- Use Burp Intruder to automate ID enumeration.

8.4 JWT Manipulation (Recap & Depth)

- **Payload tampering:** Decode header & payload, modify 'sub' or 'role', re-sign (if you know the secret) or use alg:none.
- **Key confusion:** RS256 uses private key to sign. If server uses public key as HMAC secret with HS256, forge a valid token using the public key.
- **Expiry bypass:** Some servers don't validate 'exp' claim — extend it.

8.5 Client-Side Auth Bypass

Never trust the client. Many student-level CTF apps validate credentials purely in JavaScript:

- View the source for hardcoded comparison: if (password === 'S3cr3t!') { redirect('/admin') }

- Comment out the auth check in DevTools Sources → edit the JS live.
- Use DevTools Console to call the internal function that sets the admin flag directly.
- Disable JS entirely and request the protected page directly.

8.6 HTTP Request Smuggling (Awareness)

HTTP smuggling exploits discrepancies between front-end proxy and back-end server in parsing Content-Length vs Transfer-Encoding headers. Advanced topic — study PortSwigger Web Security Academy labs for hands-on practice.

✦ Review Questions

1. What SQL payload would you use to log in as 'admin' if the password check is injectable?
2. What is IDOR and how does Burp Suite Intruder help discover IDOR vulnerabilities?
3. Why should authentication logic never be implemented purely in client-side JavaScript?
4. Explain how the JWT 'alg:none' bypass works step by step.
5. What HTTP response code or behaviour indicates a successful vertical privilege escalation attempt?

Chapter 09

Finding Flags in CTF Challenges

9.1 Flag Formats and Where to Look

Most CTFs use a standardised flag format such as flag{...}, CTF{...}, or picoCTF{...}. Knowing the format lets you search systematically.

In Binaries

- strings binary | grep -i 'flag\|ctf\|key\|pass'
- Check .rodata, .data sections in Ghidra 'Defined Strings' window.
- Dynamically: set breakpoints on strcmp, strncmp, memcmp and log arguments.

In Files / Images

- exiftool — check all metadata fields.
- binwalk -e file — extract embedded archives.
- zsteg / steghide / outguess — check LSB steganography.
- strings -n 5 file | grep -i flag

In Web Challenges

- HTML source comments:
- HTTP response headers: X-Flag or X-Secret headers.
- robots.txt or sitemap.xml entries.
- JavaScript source files (search all loaded scripts for 'flag').
- Cookie values (decode base64 cookies).
- Database error messages leaking data.

9.2 Decoding and Crypto Basics

Flags are often encoded or lightly encrypted. Master these transforms:

Encoding	Identification	Decode Method
Base64	Alphanumeric + +/= padding	base64 -d, CyberChef, Python b64decode
Hex	0-9 A-F, even length	xxd -r -p, Python bytes.fromhex()
ROT13	Readable-ish shifted letters	tr 'A-Za-z' 'N-ZA-Mn-za-m'
Caesar	Shifted alphabet, any shift	Brute force 26 shifts
XOR	Repeating byte pattern in hex	xortool, Python: bytes([a^key for a in data])
URL Encoding	%XX hex encoding	urllib.parse.unquote()
HTML Entities	< > &#xxx;	html.unescape()
Binary	48-bit blocks of 0s and 1s	int('01001000',2)

9.3 CyberChef — The Essential Decoder

CyberChef (gchq.github.io/CyberChef) is a browser-based data transformation tool with 300+ operations. Use the 'Magic' recipe to auto-detect encoding. Chain operations to handle multi-layer encoding (e.g., base64 → hex → XOR → ROT13).

9.4 CTF Category Strategies

- **Reverse Engineering:** Ghidra decompile → understand logic → patch or script a solution.
- **Crypto:** Identify cipher → find weakness (small key, reused IV, padding oracle) → exploit.
- **Forensics:** Analyse file with multiple tools; check multiple metadata layers; carve files.
- **Web:** Recon → find injection or auth flaw → extract flag from DB or filesystem.
- **Pwn/Binary Exploitation:** Find buffer overflow → control RIP → ROP chain → shell.
- **OSINT:** Use Google dorks, Shodan, Wayback Machine, WHOIS to find hidden info.

✦ Review Questions

1. Write the Linux command to search a binary for strings containing 'flag' (case-insensitive).
2. A string 'ZmxhZ3thX3NlY3JldH0=' appears in a file. What encoding is this and what does it decode to?
3. What tool would you use to auto-detect a multi-layer encoded string?
4. Where in an HTTP response other than the body might a CTF flag be hidden?
5. Name two tools to detect and extract LSB steganography from a PNG image.

Chapter 10

Essential Reverse Engineering Tools

All tools below are free and open-source unless noted. Use them only on systems you own or are authorised to test.

10.1 Disassemblers & Decompilers

Tool	Category	Description
Ghidra	Disassembler / Decompiler	NSA's free RE suite. Java-based, supports 60+ architectures, excellent decompiler, scriptable in Java/Python. *(ghidra-sre.org)*
IDA Pro	Disassembler / Decompiler	Industry gold standard. Free version (IDA Free) supports x86/x64/ARM. Hex-Rays decompiler add-on (paid). *(hex-rays.com)*
Binary Ninja	Disassembler	Modern cloud+desktop RE platform. Excellent API for scripting analysis workflows. *(binary.ninja)*
Cutter	Disassembler	GUI frontend to Radare2 + Ghidra decompiler plugin. Fully free and open-source. *(cutter.re)*
Radare2	Disassembler	Command-line RE framework. Steep learning curve but extremely powerful scripting. *(radare.org)*
RetDec	Decompiler	Open-source retargetable decompiler from Avast. Supports PE, ELF, Mach-O. *(retdec.com)*

10.2 Debuggers

Tool	Category	Description
GDB + pwndbg	Linux Debugger	GNU Debugger + pwndbg plugin for CTF-focused features: heap visualisation, ROP helpers. *(github.com/pwndbg/pwndbg)*
x64dbg	Windows Debugger	Open-source Windows user-mode debugger. Replaces OllyDbg. Excellent plugin ecosystem. *(x64dbg.com)*
WinDbg	Windows Debugger	Microsoft's kernel and user-mode debugger. Required for kernel-level RE on Windows. *(learn.microsoft.com/windbg)*
LLDB	macOS/Linux Debugger	LLVM debugger. Default on macOS. Supports Python scripting for automation. *(lldb.llvm.org)*

10.3 Web Application RE

Tool	Category	Description
Burp Suite CE	Web Proxy/Scanner	Intercept, replay, modify HTTP/S traffic. Community Edition is free and sufficient for CTFs. *(portswigger.net)*
OWASP ZAP	Web Proxy/Scanner	Free, open-source alternative to Burp. Includes active scanner. *(zaproxy.org)*

jwt_tool	JWT Analysis	Decode, tamper, sign, and brute-force JSON Web Tokens. *(github.com/ticarpi/jwt_tool)*
Nikto	Web Scanner	Fast web server scanner — finds misconfigurations and common vulnerabilities. *(cirt.net/nikto2)*
ffuf	Fuzzer	Fast web fuzzer for directory, parameter, and subdomain enumeration. *(github.com/ffuf/ffuf)*
gobuster	Directory Bruter	Directory and DNS brute-forcing tool — finds hidden paths and subdomains. *(github.com/OJ/gobuster)*

10.4 Binary Analysis & Forensics

Tool	Category	Description
binwalk	Firmware Analyser	Detect and extract embedded files, filesystems, and code from firmware/images. *(github.com/ReFirmLabs/binwalk)*
ExifTool	Metadata Reader	Read/write metadata in 150+ file formats. *(exiftool.org)*
Volatility 3	Memory Forensics	Analyse RAM dumps — extract processes, network connections, registry hives. *(volatilityfoundation.org)*
Wireshark	Packet Analyser	GUI protocol analyser. Decode hundreds of protocols; follow TCP/TLS streams. *(wireshark.org)*
Autopsy	Disk Forensics	Digital forensics platform — timeline, file carving, hash matching. *(sleuthkit.org/autopsy)*
foremost	File Carver	Carve files from raw disk images or suspicious binaries by magic bytes. *(foremost.sf.net)*

10.5 Scripting & Exploit Development

Tool	Category	Description
pwntools	Python Exploit Lib	Python library for writing CTF exploits — process/socket interaction, packing, ROP. *(docs.pwntools.com)*
angr	Symbolic Execution	Binary analysis platform; use symbolic execution to find inputs that reach targets. *(angr.io)*
ROPgadget	ROP Tool	Find Return-Oriented Programming gadgets in binaries for exploit chains. *(github.com/JonathanSalwan/ROPgadget)*
CyberChef	Data Transforms	Browser-based encoding/decoding/crypto tool — 300+ operations, chainable. *(gchq.github.io/CyberChef)*
hashcat	Password Cracker	GPU-accelerated password/hash cracker. Brute-force weak JWT secrets, MD5, SHA1. *(hashcat.net)*
John the Ripper	Password Cracker	CPU-based hash cracker. Includes zip2john, pdf2john for file format cracking. *(openwall.com/john)*

✦ Review Questions

1. What is the key difference between Ghidra and IDA Pro in terms of cost and availability?
2. Which tool would you use to analyse a RAM dump and extract running process information?
3. What is pwntools and what type of CTF challenges does it primarily assist with?
4. How does angr's symbolic execution differ from traditional dynamic debugging?
5. Name the Burp Suite feature used to automatically send the same request many times with varying payloads.

Chapter 11

Mobile App Reverse Engineering

11.1 Android APK Analysis

Android apps are distributed as APK files — ZIP archives containing compiled Java/Kotlin (classes.dex), native libraries (.so files), resources, and the AndroidManifest.xml.

Decompilation Workflow

- Step 1 — **apktool d app.apk**: Disassemble APK into Smali (readable Dalvik ASM) + resources.
- Step 2 — **jadx-gui app.apk**: Decompile to readable Java source code. Best tool for quick RE.
- Step 3 — **unzip app.apk; dex2jar classes.dex**: Convert .dex to .jar for JD-GUI.
- Step 4 — Examine AndroidManifest.xml for exported Activities, permissions, deeplinks.
- Step 5 — Search decompiled source for: API keys, hardcoded credentials, flag strings, URLs.

```
$ apktool d target.apk -o output_dir
$ jadx-gui target.apk
$ grep -r 'flag\|password\|secret\|api_key' output_dir/
```

11.2 Dynamic Analysis on Android

- **Frida:** Dynamic instrumentation toolkit. Hook Java methods at runtime — intercept encryption, bypass root detection, log arguments and return values.
- **Objection:** Frida-based tool for mobile app exploration. One command to bypass SSL pinning, root detection, and dump method calls.
- **Android Emulator (AVD):** Run APKs without a device. Use x86 images for better Frida support.
- **Drozer:** Android security assessment framework — enumerate components, test exported activities.

```
# Bypass SSL pinning with Objection:
$ frida-ps -U                    # list processes on USB device
$ objection -g com.target.app explore
>> android sslpinning disable
```

11.3 iOS IPA Analysis

- IPAs are ZIP archives. Unzip and find the Mach-O binary in Payload/AppName.app/
- Class-dump or otool reveals Objective-C class/method names.
- Ghidra and Binary Ninja support ARM64 Mach-O decompilation.
- Frida works on jailbroken iOS and Corellium virtualised devices.

✦ Review Questions

1. What is an APK file and what does it contain?
2. What tool provides the most readable Java source decompilation from an Android APK?
3. How does Frida differ from a traditional debugger in dynamic mobile RE?
4. What sensitive information would you search for in decompiled Android source code?
5. What command-line option in objection disables SSL certificate pinning?

Chapter 12

Anti-Reverse Engineering Techniques

12.1 Why Anti-RE Exists

Developers use anti-RE techniques to protect intellectual property, prevent cheating in games, and harden malware against analysis. As a reverse engineer you must recognise and defeat them.

12.2 Anti-Debugging

- **ptrace self-debug (Linux):** A process can only be ptraced by one debugger. If the program calls ptrace(PTRACE_TRACEME) first, attaching a debugger fails. Bypass: patch the ptrace call to return 0.
- **IsDebuggerPresent (Windows):** Checks PEB.NtGlobalFlag. Bypass: patch to return 0 or set the PEB flag manually.
- **Timing checks:** Measure execution time; debugger stepping is slow. Bypass: patch the time comparison or use hardware breakpoints (no timing overhead).
- **INT 3 / INT 0x2D traps:** Software breakpoints use INT 3 (0xCC). Check if byte at a known code location is 0xCC. Bypass: use hardware breakpoints (DR0-DR3).

12.3 Obfuscation

- **String encryption:** Strings are encrypted at compile time and decrypted at runtime. Bypass: break after the decryption routine runs and dump the plaintext string.
- **Control flow flattening:** All basic blocks dispatched through a central switch. Makes CFG unreadable. Bypass: use symbolic execution (angr) or trace execution.
- **Opaque predicates:** Always-true/false conditions that confuse decompilers. Identify and simplify manually.
- **Packing:** Binary is compressed/encrypted; a stub unpacks it at runtime. Detection: low entropy sections + small import table. Bypass: run, dump unpacked image (use process hacker or x64dbg's Scylla plugin).

12.4 Code Virtualisation

Advanced protections (VMProtect, Themida, Code Virtualizer) replace native x86 instructions with a custom virtual machine bytecode. Extremely difficult to RE. Approaches: trace VM handlers, identify patterns in the dispatch loop, or use symbolic execution to model VM semantics. Beyond scope of most CTFs but common in professional malware analysis.

12.5 Packers — Detect and Unpack

- Run 'Detect It Easy' (DIE) or 'PEiD' to identify known packers.
- Entropy analysis: packed sections have entropy > 7.0 (near-random).
- Generic unpack: set breakpoints on VirtualAlloc + WriteProcessMemory; dump memory when execution transfers to the unpacked code (OEP).

```
# Check entropy with Python:
import math, collections
data = open('packed.exe','rb').read()
counts = collections.Counter(data)
```

```
entropy = -sum((c/len(data))*math.log2(c/len(data)) for c in counts.values())
print(f'Entropy: {entropy:.2f}')  # >7.0 = likely packed
```

✦ Review Questions

1. How does a program use ptrace() to detect and block a Linux debugger?
2. What are hardware breakpoints and why do they bypass INT 3 detection?
3. What is binary packing and what entropy value indicates a packed section?
4. How would you bypass a runtime string decryption routine to read the plaintext?
5. What tool would you use to identify which packer was used on a Windows PE binary?

Chapter 13

Network Traffic Analysis

13.1 PCAP Analysis with Wireshark

A PCAP (packet capture) file stores raw network traffic. CTF forensics challenges frequently provide PCAPs with flags hidden in the traffic.

- **File → Open:** Load .pcap or .pcapng file.
- **Follow TCP Stream:** Right-click a packet → Follow → TCP Stream. Reassembles full conversation as readable text.
- **Export HTTP Objects:** File → Export Objects → HTTP. Extracts all files transferred over HTTP (images, scripts, archives).
- **Display filters:** http, dns, tcp.port==443, ip.addr==10.0.0.1.
- **Statistics → Protocol Hierarchy:** See which protocols dominate the capture.

```
# Command-line PCAP analysis with tshark:
$ tshark -r capture.pcap -Y 'http' -T fields -e http.request.uri
$ tshark -r capture.pcap -Y 'dns' -T fields -e dns.qry.name
```

13.2 DNS Exfiltration

Data can be covertly exfiltrated by encoding it in DNS query hostnames (e.g., flag_part1.attacker.com). In Wireshark, filter for 'dns' and examine unusually long or base64-like subdomain names.

13.3 TLS Decryption

Modern HTTPS uses TLS encryption. To decrypt captured TLS traffic:

- Set SSLKEYLOGFILE environment variable before starting the browser.
- In Wireshark: Preferences → Protocols → TLS → (Pre)-Master-Secret log file.
- Now Wireshark can decrypt and show HTTP/2 plaintext.

13.4 Protocol Reverse Engineering

Custom binary protocols appear in CTFs and malware analysis. Approach:

- Observe fixed headers — magic bytes, version fields, length fields.
- Vary one input at a time and observe which bytes change.
- Use Wireshark's 'Decode As' to force a known protocol decoder.
- Scapy (Python) lets you craft and send custom protocol packets.

✦ Review Questions

1. How do you reassemble a complete HTTP conversation in Wireshark?
2. What Wireshark feature extracts all files transferred over HTTP from a PCAP?
3. What environment variable instructs browsers to log TLS session keys for decryption?
4. Describe how data can be exfiltrated via DNS and how you would detect it in a PCAP.
5. What is the tshark command to extract all DNS query names from a capture file?

Chapter 14

Binary Exploitation Basics

This chapter introduces fundamental exploitation concepts. Practise exclusively on CTF platforms and intentionally vulnerable VMs (pwn.college, exploit.education, protostar).

14.1 Buffer Overflow

A buffer overflow occurs when a program writes more data to a fixed-size buffer than it can hold, overwriting adjacent memory including the saved return address on the stack.

Classic Stack Overflow

```
char buf[64];
gets(buf);          // reads unlimited input — DANGEROUS
```

If you input 72 bytes of 'A' + 8 bytes of a target address, the return address is overwritten and execution redirects to your chosen address.

Finding the Offset

- Generate a cyclic pattern: python3 -c "from pwn import *; print(cyclic(200))"
- Run in GDB, read the value in RSP/RIP at crash, calculate offset with cyclic_find().

14.2 Memory Protections

Protection	Description	Bypass Technique
ASLR	Randomises base addresses each run	Leak a pointer, compute base = leak - known_offset
NX / DEP	Stack/heap memory not executable	ROP (Return-Oriented Programming) — reuse existing code gadgets
Stack Canary	Random value before return address; checked on return	Leak the canary, include it in overflow payload
PIE	Binary itself loaded at random address	Leak binary address; compute offset to target function
RELRO	Makes GOT read-only	Use other write primitives or pre-initialised values

14.3 Return-Oriented Programming (ROP)

ROP chains together short instruction sequences ending in 'ret' (called gadgets) to perform arbitrary computation without injecting new code. Each gadget pops arguments from the stack before returning to the next gadget.

- Find gadgets: ROPgadget --binary ./target
- Common gadgets: 'pop rdi; ret' (set argument), 'pop rsi; ret', 'ret' (stack alignment).
- Goal: call system('/bin/sh') by controlling RDI and locating system() via libc leak.

14.4 Format String Vulnerabilities

printf(user_input) — if user_input contains format specifiers, the attacker can read and write arbitrary memory:

- %x / %p — print stack values as hex (leak addresses, canaries).
- %s — dereference pointer from stack and print as string.
- %n — write the number of bytes printed so far to an address. Arbitrary write primitive.

```
printf("%7$p")    // print the 7th argument (stack value) as pointer
```

✦ Review Questions

1. What is a buffer overflow and what critical stack value is typically targeted?
2. How do you determine the exact byte offset to the return address in a buffer overflow?
3. What is Return-Oriented Programming and why is it needed when NX/DEP is enabled?
4. What does the format string specifier %n do, and why is it dangerous?
5. List three binary security mitigations and describe a general bypass strategy for each.

Chapter 15

Practice Platforms & Learning Path

15.1 Recommended CTF Platforms

Platform	Focus Areas	URL
picoCTF	Beginner-friendly: RE, crypto, web, forensics	picoctf.org
HackTheBox	Intermediate: machines, challenges, pro labs	hackthebox.com
TryHackMe	Guided learning paths: beginner to advanced	tryhackme.com
pwn.college	Binary exploitation & assembly deep dive	pwn.college
CryptHack	Cryptography — from basics to advanced	cryptohack.org
PortSwigger Academy	Web security — 250+ free labs	portswigger.net/web-security
crackmes.one	RE-focused binary crackme challenges	crackmes.one
exploit.education	Vulnerable VMs for pwn practice	exploit.education
CTFtime	Archive of all past CTF competitions	ctftime.org
RingZer0	Diverse challenges: coding, RE, crypto	ringzer0ctf.com

15.2 Recommended Books

- **Hacking: The Art of Exploitation** — Jon Erickson. Best intro to binary exploitation and shellcode.
- **The Shellcoder's Handbook** — Koziol et al. Comprehensive coverage of vulnerability classes.
- **Practical Malware Analysis** — Sikorski & Honig. Industry-standard malware RE guide.
- **Reversing: Secrets of Reverse Engineering** — Eldad Eilam. Foundational RE concepts.
- **The Web Application Hacker's Handbook** — Stuttard & Pinto. Comprehensive web RE/hacking.
- **Hacking APIs** — Corey Ball. Modern API security testing guide.

15.3 Suggested Learning Path

Level	Duration	Topics	Milestone
Foundations	Months 1-2	Linux CLI, Python scripting, networking basics, x86 ASM intro	Complete 10 picoCTF challenges
Beginner RE	Months 3-4	Ghidra, GDB, static/dynamic analysis, ELF format	Solve 5 crackmes on crackmes.one
Web RE	Months 5-6	Burp Suite, JS deobfuscation, SQLi, JWT attacks, PortSwigger labs	Complete PortSwigger SQLi & Auth modules
Intermediate	Months 7-9	Buffer overflows, ROP, format strings, mobile RE	Solve HackTheBox Easy machines
Advanced	Months 10-12	Kernel exploitation, heap exploitation, malware analysis, pro labs	Hack The Box Pro Labs or OSCP prep

15.4 Professional Certifications

- **eJPT** (eLearnSecurity Junior Penetration Tester) — entry-level, practical, affordable.
- **CEH** (Certified Ethical Hacker) — broad coverage, industry recognition, MCQ-heavy.
- **OSCP** (Offensive Security Certified Professional) — gold standard, 24-hour practical exam.
- **GREM** (GIAC Reverse Engineering Malware) — specialist RE and malware analysis cert.
- **CRTO** (Certified Red Team Operator) — Cobalt Strike + red team operations.

15.5 Final Advice for Students

- Build a lab: Kali Linux VM + Windows VM + intentionally vulnerable targets.
- Keep a notes repository (Obsidian, Notion) — document every technique and write-up.
- Engage with the community: r/netsec, CTF Discord servers, local DEFCON chapters.
- Contribute to open-source tools — reading tool source code is an education in itself.
- Approach every challenge as a puzzle — frustration is a sign you are learning.
- Always act with integrity: the skills in these notes carry legal and ethical responsibility.

✦ Review Questions

1. Which platform would you recommend for a complete beginner starting CTF challenges?
2. Name two platforms focused specifically on web application security practice.
3. What certification is considered the gold standard for penetration testing?
4. What should you document in a personal security notes repository after each challenge?
5. In your own words, describe the most important ethical principle for a cybersecurity student.

Knowledge is neutral — intent and authorisation define the difference between security research and cybercrime. Always obtain written permission. Always act with integrity. Good luck in your studies and CTF competitions.